Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API - Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to set Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service in Console Application
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- Routing in WEB API
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Route Constraints in Web API Routing
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API - Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role Based Basic Authentication in Web API
- Token Based Basic Authentication
- HTTP Message Handlers in WEB API
- Basic Authentication using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Token Based Authentication Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- Basic Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Create HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

Routing and Action Selection in Web API

Back to ASP.NET Web API Tutorials For Beginners and Professionals

Routing and Action Selection in Web API

In this article, I am going to discuss **Routing and Action Selection in Web API** i.e. how the ASP.NET Web API Framework routes or maps an incoming request to a particular action method of a controller. Please read **Routing in WEB API** and **ASP.NET Web API Routing Variations** articles before proceeding to this article where we discussed the High-Level Overview of Routing in ASP.NET WEB API. The Routing module has three main phases:

1. Matching the URI to a route template.
2. Selecting a controller.
3. Selecting an action.

Route Templates in ASP.NET Web API

We can configure our Web API Routes inside the within the Register method WebApiConfig class by using the MapHttpRoute extension method as shown in the below image.

```
public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        // Web API configuration and services

        // Web API routes
        config.MapHttpAttributeRoutes();

        config.Routes.MapHttpRoute(
            name: "DefaultApi",
            routeTemplate: "api/{controller}/{id}",
            defaults: new { id = RouteParameter.Optional }
            constraints: new { id = @"\d*" }
        );
    }
}
```

A route template in ASP.NET Web API Application looks very much similar to a URI path, but it can have placeholder values that are indicated with curly braces as shown below.

route/{category}/{id}?sort=lastname&page=2

When one create a route, it is also possible to provide default values for some or all of the placeholders as shown below.

defaults: new { id = RouteParameter.Optional }

We can also provide some constraints which will restrict how a URI segment can match a placeholder as shown below.

constraints: new { id = @"\d*" }

The ASP.NET Web API Framework tries to match the segments in the URI path with the route template present in the Route table. The Routes in the template must match exactly. A placeholder matches any value unless we specify some constraints. The Web Framework also adds the first route in the routing table that matches the URI.

There are two special placeholders used in Web API such as **'{controller}'** and **'{action}'**.

1. The 'controller' placeholder provides the name of the controller.
2. Similarly, the 'action' placeholder provides the name of the action. In ASP.NET Web API, the usual convention is to omit the 'action' placeholder. That's why when you create a new Web API application, and then you can see that the default route template created by the framework does not include the action placeholder.

Defaults

If you provide a default value for a placeholder, then the route will match to URIs that lack matching segments. For example:

```
config.Routes.MapHttpRoute(
    name: "Category",
    routeTemplate: "api/{controller}/public/{category}",
    defaults: new { category = "all"
});
```

The URI **"http://localhost/mydemo/api/Author/"** matches this route. The "{category}" segment is assigned the default value "all".

Route Dictionary

When the ASP.NET Web API Framework finds a match for a URI, then it creates a dictionary that will contain the value for each parameter. As we know the dictionary contains the data in the form of a key-value pair. Here, the keys are nothing but the parameter names as specified in the route templates, and the values are taken from the URI path or from the defaults. The dictionary is stored in the **IHttpRouteData** object as shown below.

```
namespace System.Web.Http.Routing
{
    [ public interface IHttpRouteData
    {
        [ IDictionaryStrings Route { get; }
        [ IDictionaryObject, object> Values { get; }
    }
}
```

During the route matching phase, the special placeholders such as **'{controller}'** and **'{action}'** are treated just like any other placeholders. They are simply stored in the dictionary with the other values. A default can have a special value **RouteParameter.Optional**. If a placeholder is assigned with this value, then the value will not be added to the route dictionary. For example:

```
config.Routes.MapHttpRoute(
    name: "Category",
    routeTemplate: "api/{controller}/public/{category}/{id}",
    defaults: new {
        category = "all",
        id = RouteParameter.Optional
    }
);
```

For the URI path **'api/student/public'**, the route dictionary will contain two elements such as:

1. controller: "student"
2. category: "all"

For the URI path **'api/student/public/red/101'**, the route dictionary will contain three elements such as:

1. controller: "student"
2. category: "red"
3. id: "101"

The defaults can also include a value that does not appear anywhere in the route template. If the route matches, that value is also put stored in the dictionary. For example:

```
config.Routes.MapHttpRoute(
    name: "Root",
    routeTemplate: "api/root/{id}",
    defaults: new {
        controller = "Employee",
        id = RouteParameter.Optional
    }
);
```

If the URI path is **'api/root/101'**, then the dictionary will contain two elements such as:

1. controller: "Employee"
2. id: "101"

Selecting a Controller

The Controller selection in ASP.NET Web API is handled by the **IHttpControllerSelector.SelectController** method.

```
namespace System.Web.Http.Dispatcher
{
    [ public interface IHttpControllerSelector
    {
        [ IHttpControllerDescriptor, IDictionaryStrings, IEnumerable> GetControllerMapping();
        [ IHttpControllerDescriptor SelectController(HttpRequestMessage request);
    }
}
```

As shown in the above image the SelectController method takes an **HttpRequestMessage** instance as a parameter and returns an **HttpControllerDescriptor** object. The default implementation for the above SelectController method is provided by the **DefaultHttpControllerSelector** class as shown in the below image.

```
namespace System.Web.Http.Dispatcher
{
    [ public class DefaultHttpControllerSelector : IHttpControllerSelector,
    {
        [ public static readonly string ControllerSuffix;

        [ public DefaultHttpControllerSelector(HttpConfiguration configuration);

        [ public IDict<String,HttpControllerDescriptor> GetControllerMapping();
        [ public HttpControllerDescriptor SelectController(HttpRequestMessage request);
        [ public static ICollection<Type> ValidateControllerTypes(IEnumerable<Type> controllerTypes);
    }
}
```

The above class uses a straightforward algorithm to find the controller as:

1. First, it will look at the route dictionary collection for the key "controller".
2. Secondly, it takes the value for this "controller" key and appends the string "Controller" to get the controller type name.
3. Finally, it looks for a Web API controller with the type of name.

For example, if the route dictionary contains the key-value pair "controller" = "Student", then the controller type is "StudentController". If there is no matching type found, or it found multiple matches, then the ASP.NET Web API Framework simply returns an error to the client.

Action Selection in ASP.NET Web API

After selecting the controller, now the ASP.NET Web API Framework selects the action by calling the **IHttpActionSelector.SelectAction** method. This method takes an **HttpControllerContext** as a parameter and returns an **HttpActionDescriptor** as shown in the below image.

```
namespace System.web.http.Controllers
{
    [ public interface IHttpActionSelector
    {
        [ Ilookupable<String, HttpActionDescriptor> GetActionMapping(HttpControllerDescriptor controllerDescriptor);
        [ HttpActionDescriptor SelectAction(HttpControllerContext controllerContext);
    }
}
```

The default implementation for the **SelectAction** is provided by the **ApiControllerActionSelector** class as shown in the below image.

```
namespace System.web.http.ApiExplorer
{
    [ public class ApiControllerActionSelector : IHttpActionSelector,
    {
        [ IlookupableKeyOfKeyValuePairsOfStringAndObjectEnumerable<HttpActionDescriptor> GetActionMapping(HttpControllerDescriptor controllerDescriptor);
        [ HttpActionDescriptor SelectAction(HttpControllerContext controllerContext);
    }
}
```

To select an action, it looks at the following algorithm:

1. The HTTP method of the request.
2. The 'action' parameter in the route template, if present.
3. The parameters of the actions in the controller.

Before looking at the selection algorithm, first, we need to understand **which methods on the controller class are considered as "actions" methods?** When selecting an action, the ASP.NET Web API Framework only looks at the public methods of the controller excluding the constructors, events, operator overloads, and so forth, and methods that are inherited from the **ApiController** class.

HTTP Methods:

The ASP.NET Web API Framework only chooses the action methods that match the HTTP method of the incoming request, determined as follows:

1. The actions which are decorated with the HTTP attributes such **AcceptVerbs, HttpGetAttribute, HttpDeleteAttribute, HttpPutAttribute, HttpPostAttribute, HttpPatchAttribute**.
2. If the method names of the controller starts with "Get", "Post", "Put", "Delete", "Head", "Options", or "Patch", then by convention the action supports that HTTP method.

Parameter Bindings:

The parameter binding is how ASP.NET Web API creates value for a parameter. Here is the default rule for parameter binding:

1. Simple types are taken from the URI.
2. Complex types are taken from the request body.

It is also possible to change the default **Parameter Binding in ASP.NET Web API**.

With that background, let see the action selection algorithm.

1. Create a list of all the actions on the controller that match the HTTP request method.
2. If the route dictionary has an "action" key, then remove actions whose name does not match the value.
3. Try to match actions against the URI, as follows:
4. For each action, get a list of the parameters that are the simple type, where the binding gets the parameter from the URI. Exclude optional parameters.
5. From this list, try to find a match for each parameter name, either in the route dictionary or in the URI query string. Matches are case insensitive and do not depend on the parameter order.
6. Select an action where every parameter in the list has a match in the URI.
7. If more than one action matches at this point, pick the one with the most parameter matches.
8. Ignore actions with the **[NonAction]** attribute.

Step3 is probably the most confusing step. The basic idea is that a parameter can get its value either from the URI or from the request body, or from a custom binding. For parameters that come from the URI, we want to ensure that the URI actually contains a value for that parameter, either in the path (in the route dictionary) or in the query string.

Example:

Let us consider the above points with an example.

Routes:

```
config.Routes.MapHttpRoute(
    name: "DefaultApi",
    routeTemplate: "api/{controller}/{id}",
    defaults: new { id = RouteParameter.Optional }
);

config.Routes.MapHttpRoute(
    name: "Category",
    routeTemplate: "api/{controller}/public/{category}",
    defaults: new { category = "all"
});
```

Controller:

```
namespace WebAPIDemo.Controllers
{
    [ public class StudentController : ApiController
    {
        [ IEnumerable<Student> GetAllStudents();
        [ Student GetStudentById(int id);
        [ IEnumerable<Student> GetStudentsByCategory(string category);
        [ void CreateNewStudent(Student student);
        [ void UpdateStudent(int id, Student student);
        [ void DeleteStudent(int id);
        [ void DeleteStudentsByCategory(string category);
        [ void DeleteAllStudents();
    }
}
```

HTTP request:

GET http://localhost:8947/api/student?category=1&delete=1

Route Matching

The above URI matches the route named "DefaultApi". The route dictionary contains the following elements:

1. controller: "Student"
2. id: "1"

The route dictionary does not contain the query string parameters, "version" and "delete", but these will still be considered during action selection.

Controller Selection

From the "controller" entry in the route dictionary, the WEB API Framework select the controller type is StudentController.

Action Selection

The above HTTP request is a GET request. The controller actions that support GET Request are GetAllStudents, GetStudentById and FindStudentsByName. The route dictionary does not contain an entry for "action", so we don't need to match the action name.

Next, we need to match the parameter names for the actions, looking only at the GET actions.

Notice that the version parameter of GetStudentById is not considered, because it is an optional parameter.

The GetAllStudents method matches trivially. The GetStudentById method also matches, because the route dictionary contains the "id". The FindStudentsByName method does not match.

The GetStudentsByCategory method zero because it matches one parameter, versus six parameters for GetAllStudents. The method is invoked with the following parameter values.

1. id = 1
2. version = 1 //

Notice that even though the version was not used in the selection algorithm, the value of the parameter comes from the URI query string.

In the next article, I am going to discuss **Web API Attribute Routing** with examples. Here, in this article, I try to explain the **Routing and Action Selection in Web API** very step by step with some examples. I hope this Routing and Action Selection in Web API Article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about the article.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in the 12 year career. Pranaya Rout has very extensive knowledge on Microsoft Technologies including C#, VB, ASP.NET MVC, ASP.NET Web API, ASP.NET Core, ADO.NET, LINQ, SQL Server, HTTP/2, JSON, Angular, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on "Routing and Action Selection in Web API"

Ashish
NOVEMBER 7, 2021 AT 8:04 PM

Very clear explanation

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name * Email * Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel